ASSIGNMENT – 11.3

HT.NO: 2303A52498

BATCH: 50

TASK – 01:

Question:

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches: o Add a contact o Search for a contact o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:

o Insertion efficiency o

Deletion efficiency

Prompt:

Generate a python program for a Smart Contact Manager.

Implement it using Arrays and Linked Lists.

Each contact should store name and phone number.

Include operations:

Add contact

Search contact

Delete contact

Use menu-driven user input and keep the code simple for beginners.

Also add a short comparison of arrays vs linked lists for insertion and deletion.

Generated Code:

```
class Contact:     def __init__(self,
name, phone):
        self.name = name         self.phone =
phone class Node:     def __init__(self,
contact):         self.contact = contact
self.next = None class ContactManager:
def __init__(self):         self.head = None
def add_contact(self, name, phone):
new_contact = Contact(name, phone)
new_node = Node(new_contact)
        if not self.head:
            self.head = new_node
            return
current     =     self.head
while current.next:
            current = current.next
current.next = new_node     def
search_contact(self, name):
        current  =  self.head                 while
current:                 if current.contact.name
==  name:                                  return
current.contact.phone                 current =
current.next                 return  "Contact  not
found."    def delete_contact(self, name):
        current = self.head
previous      =      None
while current:
            if current.contact.name == name:
                if previous:
```

```
                previous.next = current.next
            else:
                self.head = current.next
return "Contact deleted."
previous = current            current =
current.next        return "Contact
not found." def main():
   manager = ContactManager()
while True:
      print("\nSmart Contact Manager")
print("1. Add Contact")        print("2.
Search Contact")        print("3. Delete
Contact")        print("4. Exit")
choice = input("Enter your choice: ")
if choice == '1':
          name = input("Enter contact name: ")
phone = input("Enter contact phone number: ")
manager.add_contact(name, phone)
print("Contact added.")        elif choice == '2':
          name = input("Enter contact name to search: ")
result = manager.search_contact(name)
print(f"Phone number: {result}")        elif choice ==
'3':
          name = input("Enter contact name to delete: ")
result = manager.delete_contact(name)
          print(result)
elif choice == '4':
print("Exiting...")
break        else:
```

```
        print("Invalid choice. Please try again.") if
__name__ == "__main__":
    main()
"""
```

Comparison of Arrays vs Linked Lists for Insertion and Deletion:

1. Insertion:

- Arrays: Inserting an element in the middle of an array requires shifting all subsequent elements, which can be time-consuming (O(n) in the worst case).

- Linked Lists: Inserting an element in a linked list is more efficient as it only requires updating a few pointers (O(1) if inserting at the beginning or end).

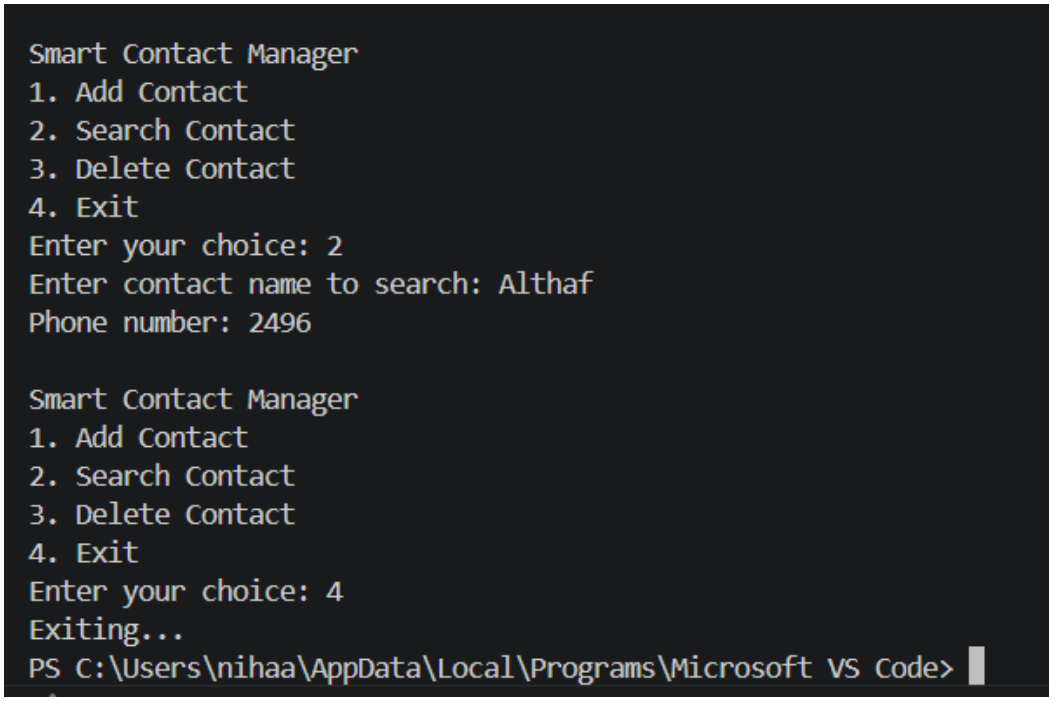
2. Deletion:

- Arrays: Deleting an element from an array also requires shifting all subsequent elements to fill the gap, which can be time-consuming (O(n) in the worst case).

- Linked Lists: Deleting an element from a linked list is more efficient as it only requires updating a few pointers (O(1) if deleting the first or last element).

Overall, linked lists are more efficient for insertion and deletion operations compared to arrays, especially when dealing with large datasets or when the size of the dataset is not known in advance. However, arrays provide faster access to elements due to their contiguous memory allocation, making them more suitable for scenarios where random access is required."""

OUTPUT:



Explanation:

This program implements a Smart Contact Manager using a linked list in Python. It allows users to add, search, and delete contacts, where each contact stores a name and phone number. The menu-driven main() function lets the user interact with these operations until they choose to exit.

TASK – 02:

Question:

The SRU Library manages book borrow requests. Students and faculty submit requests,

but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.

2. Extend the system to a Priority Queue, prioritizing faculty requests.

3. Use GitHub Copilot to assist in generating:

o enqueue() method o

dequeue() method

4. Test the system with a mix of student and faculty requests.

Prompt:

Write a simple python program for a Library Book Request System.

Use:

Queue (FIFO) for book requests

Priority Queue where faculty requests are processed before student requests

Each request should include name and role (student/faculty).

Implement enqueue() and dequeue() methods and keep the code very simple for beginners.

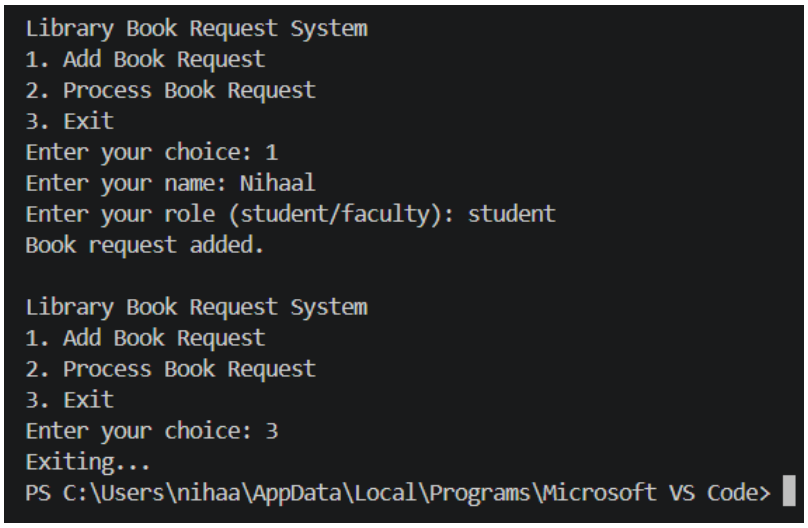
Generated Code:

```
class BookRequest:
    def __init__(self, name, role):
        self.name = name
self.role = role
```

```
class LibraryRequestSystem:     def
__init__(self):         self.queue = []
def enqueue(self, name, role):
request = BookRequest(name, role)
self.queue.append(request)     def
dequeue(self):         if not self.queue:
            return "No requests in the queue."
# Process faculty requests first         for i
in range(len(self.queue)):             if
self.queue[i].role == 'faculty':
                return self.queue.pop(i)
        # If no faculty requests, process student requests
return self.queue.pop(0) def main():

    system = LibraryRequestSystem()
while True:
        print("\nLibrary Book Request System")
print("1. Add Book Request")         print("2.
Process Book Request")         print("3. Exit")
choice = input("Enter your choice: ")         if
choice == '1':
            name = input("Enter your name: ")             role =
input("Enter your role (student/faculty): ")
system.enqueue(name, role)             print("Book request
added.")         elif choice == '2':
            result = system.dequeue()
if isinstance(result, BookRequest):
                print(f"Processing request for {result.name} ({result.role}).")
            else:
```

```
            print(result)
elif choice == '3':
print("Exiting...")
break        else:
        print("Invalid choice. Please try again.") if
__name__ == "__main__":    main()
```

Output:



Explanation:

This program implements a Library Book Request System using a queue. Users can add book requests with their name and role (student or faculty). When processing requests, faculty requests are given priority over student requests.

TASK – 03:

Question:

SR University's IT Help Desk receives technical support tickets from students and

staff. While tickets are received sequentially, issue escalation follows a

Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.

2. Provide the following operations: o push(ticket) o pop() o peek()

3. Simulate at least five tickets being raised and resolved.

4. Use GitHub Copilot to suggest additional stack operations such as:

o Checking whether the stack is empty

o Checking whether the stack is full (if applicable)

Prompt:

Write a simple python program for an IT Help Desk Ticket System using a Stack (LIFO).

Each ticket should contain ticket ID and issue description.

Implement operations: push() to

add a ticket pop() to resolve a

ticket peek() to view the latest

ticket check if the stack is empty

or full

Simulate at least five tickets and keep the code simple with comments for beginners.

Generated Code:

```
class Ticket:     def __init__(self, ticket_id,
description):
        self.ticket_id = ticket_id
self.description = description class
HelpDesk:
    def __init__(self, capacity):
self.stack = []         self.capacity =
capacity     def push(self, ticket_id,
description):         if len(self.stack) >=
self.capacity:
            print("Stack is full. Cannot add more tickets.")
            return         ticket =
Ticket(ticket_id, description)
self.stack.append(ticket)
print(f"Ticket {ticket_id} added.")     def
pop(self):         if not self.stack:
```

```
        print("Stack is empty. No tickets to resolve.")
        return        ticket = self.stack.pop()        print(f"Resolving
ticket {ticket.ticket_id}: {ticket.description}")    def peek(self):
if not self.stack:
        print("Stack is empty. No tickets to view.")
        return        ticket = self.stack[-1]        print(f"Latest
ticket {ticket.ticket_id}: {ticket.description}")    def
is_empty(self):
    return len(self.stack) == 0
def is_full(self):
    return len(self.stack) >= self.capacity def
main():
  help_desk = HelpDesk(capacity=5)    # Simulating
ticket creation    help_desk.push(1, "Cannot connect
to Wi-Fi.")    help_desk.push(2, "Computer is running
slow.")    help_desk.push(3, "Software installation
request.")    help_desk.push(4, "Printer not working.")
help_desk.push(5, "Email not syncing.")    #
Attempting to add another ticket when stack is full
help_desk.push(6, "Keyboard not responding.")
  # Viewing the latest ticket
help_desk.peek()    #
Resolving tickets
help_desk.pop()
help_desk.pop()
  # Checking if stack is empty
  print("Is the stack empty?", help_desk.is_empty())
```

Resolving remaining tickets help_desk.pop()

help_desk.pop() help_desk.pop() # Attempting to pop

from an empty stack

Checking if stack is empty print("Is the stack

empty?", help_desk.is_empty()) if __name__ ==

"__main__":

main() **Output:**

```
PS C:\Users\nihaa\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe
Ticket 1 added.
Ticket 2 added.
Ticket 3 added.
Ticket 4 added.
Ticket 5 added.
Stack is full. Cannot add more tickets.
Latest ticket 5: Email not syncing.
Resolving ticket 5: Email not syncing.
Resolving ticket 4: Printer not working.
Is the stack empty? False
Resolving ticket 3: Software installation request.
Resolving ticket 2: Computer is running slow.
Resolving ticket 1: Cannot connect to Wi-Fi.
Stack is empty. No tickets to resolve.
Is the stack empty? True
```

Explanation:

This program simulates a Help Desk Ticket System using a stack (LIFO – Last In, First Out). Tickets are added with an ID and description, and the most recently added ticket is resolved first. It also checks if the stack is full or empty and allows viewing the latest ticket using peek().

TASK – 04:

Question:

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

• Insert

• Search

• Delete Starter Code class HashTable:

pass

Prompt:

Generate a simple Hash Table program with operations:

Insert

Search

Delete

Use collision handling with chaining.

Start with this code: class

HashTable:

pass

Write simple python code, with well-commented methods."""

class HashTable:

Generated Code:

```
class HashTable:     def __init__(self, size=10):
        # Initialize the hash table with a specified size          self.size = size
self.table = [[] for _ in range(size)]  # Create a list of empty lists for chaining
def hash_function(self, key):
        # Simple hash function to calculate the index for a given key
return hash(key) % self.size      def insert(self, key, value):
        # Insert a key-value pair into the hash table
index = self.hash_function(key)

        # Check if the key already exists in the chain at the index
for i, (k, v) in enumerate(self.table[index]):              if k == key:
self.table[index][i] = (key, value)  # Update existing key
                return
        self.table[index].append((key, value))  # Add new key-value pair
def search(self, key):
```

```
        # Search for a value by its key in the hash table
index = self.hash_function(key)         for k, v in
self.table[index]:              if k == key:
                return v  # Return the value if key is found
return None  # Return None if key is not found      def
delete(self, key):
        # Delete a key-value pair from the hash table
index = self.hash_function(key)         for i, (k, v)
in enumerate(self.table[index]):              if k ==
key:
                del self.table[index][i]  # Remove the key-value pair from the chain
return True  # Return True if deletion is successful          return False  #
Return False if key is not found def main():
    hash_table = HashTable()
    # Inserting key-value pairs into the hash table
hash_table.insert("name", "Alice")     hash_table.insert("age",
30)     hash_table.insert("city", "New York")     # Searching
for values by keys     print("Name:",
hash_table.search("name"))  # Output: Alice print("Age:",
hash_table.search("age"))    # Output: 30 print("City:",
hash_table.search("city"))  # Output: New York
print("Country:", hash_table.search("country"))  # Output:
None (not found)
    # Deleting a key-value pair     print("Deleting 'age':",
hash_table.delete("age"))  # Output: True
    print("Searching for 'age' after deletion:", hash_table.search("age"))  # Output: None (not
found) if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\nihaa\AppData\Local\Programs\Microsoft VS Code> &
Name: Alice
Age: 30
City: New York
Country: None
Deleting 'age': True
Searching 'age' after deletion: None
```

Explanation:

This program implements a Hash Table in Python to store key–value pairs using a hash function. It supports insert, search, and delete operations, and handles collisions using chaining (lists at each index). The main() function demonstrates adding, searching, and deleting data from the hash table.

TASK – 05:

Question:

Design a Campus Resource Management System with the following features:

• Student Attendance Tracking

• Event Registration System

• Library Book Borrowing

• Bus Scheduling System

• Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.

2. Justify your choice in 2–3 sentences.

3. Implement one selected feature using AI-assisted code generation. **Prompt:**

Design a Campus Resource Management System with the following features:

• Student Attendance Tracking

• Event Registration System

• Library Book Borrowing

• Bus Scheduling System

• Cafeteria Order Queue

1. Choose the most appropriate data structure for each feature. write python code snippets for each feature using the chosen data structure. and print outputs for each feature to demonstrate their functionality.

2. Justify your choice in 2–3 sentences.

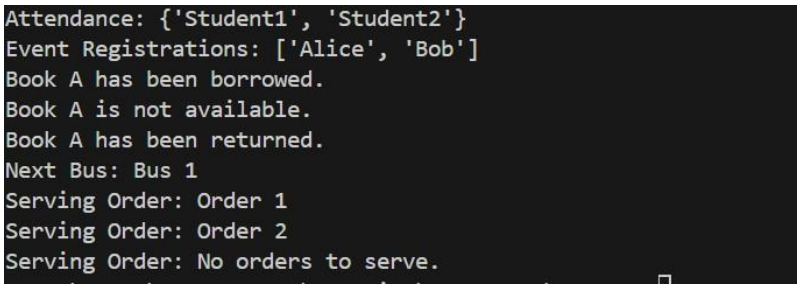
Generated Code:

```
# Student Attendance Tracking - Using a Set to store unique student IDs
class AttendanceTracker:     def __init__(self):
        self.attendance = set()  # Using a set to store unique student IDs
def mark_attendance(self, student_id):
        self.attendance.add(student_id)  # Add student ID to the set
def get_attendance(self):
        return self.attendance  # Return the set of attended student IDs #
Event Registration System - Using a List to store event registrations
class EventRegistration:     def __init__(self):
        self.registrations = []  # Using a list to store event registrations
def register(self, name):
        self.registrations.append(name)  # Add participant name to the list
    def get_registrations(self):
        return self.registrations  # Return the list of registered participants #
Library Book Borrowing - Using a Dictionary to store book availability
class Library:     def __init__(self):
        self.books = {"Book A": True, "Book B": True, "Book C": True}  # Using a dictionary to
store book availability     def borrow_book(self, book_name):         if
self.books.get(book_name, False):
            self.books[book_name] = False  # Mark the book as borrowed
return f"{book_name} has been borrowed."         return
f"{book_name} is not available."     def return_book(self,
book_name):         if book_name in self.books:
            self.books[book_name] = True  # Mark the book as available
return f"{book_name} has been returned."         return
f"{book_name} does not exist in the library."
```

```
# Bus Scheduling System - Using a Queue to manage bus schedules
from collections import deque class BusSchedule:     def
__init__(self):
        self.schedule = deque()  # Using a queue to manage bus schedules
def add_bus(self, bus_number):
        self.schedule.append(bus_number)  # Add bus number to the schedule
def next_bus(self):         if self.schedule:
            return self.schedule.popleft()  # Get the next bus in the schedule
return "No buses scheduled."
# Cafeteria Order Queue - Using a Queue to manage cafeteria orders
class CafeteriaOrderQueue:
def __init__(self):
        self.orders = deque()  # Using a queue to manage cafeteria orders
def place_order(self, order):
        self.orders.append(order)  # Add order to the queue
def serve_order(self):         if self.orders:
            return self.orders.popleft()  # Serve the next order in the queue
return "No orders to serve." def main():
    # Demonstrating Student Attendance Tracking
attendance_tracker = AttendanceTracker()
attendance_tracker.mark_attendance("Student1")
attendance_tracker.mark_attendance("Student2")
print("Attendance:", attendance_tracker.get_attendance())

    # Demonstrating Event Registration System     event_registration
= EventRegistration()     event_registration.register("Alice")
event_registration.register("Bob")     print("Event Registrations:",
event_registration.get_registrations())
```

```
    # Demonstrating Library Book Borrowing                library = Library()
print(library.borrow_book("Book A"))      print(library.borrow_book("Book A"))  #
Attempt to borrow the same book again      print(library.return_book("Book A"))


    # Demonstrating Bus Scheduling System
    bus_schedule = BusSchedule()
bus_schedule.add_bus("Bus 1")
bus_schedule.add_bus("Bus 2")     print("Next
Bus:", bus_schedule.next_bus())


    # Demonstrating Cafeteria Order Queue
cafeteria_queue = CafeteriaOrderQueue()
cafeteria_queue.place_order("Order 1")
cafeteria_queue.place_order("Order 2")     print("Serving
Order:", cafeteria_queue.serve_order())     print("Serving
Order:", cafeteria_queue.serve_order())
    print("Serving Order:", cafeteria_queue.serve_order())  # Attempt to serve when no orders
are left if __name__ == "__main__":    main()
```

Output:



Explanation:

This program demonstrates different Python data structures for real-life systems: a set for student attendance, a list for event registrations, a dictionary for library book availability, and queues (deque) for bus scheduling and cafeteria orders. The main() function shows how each system works by adding, retrieving, and processing data.